Contradiction Finder for Research Reading: A Web-Based Assistant for Surfacing Possible Scientific Disagreements

Sean Liao
University of Illinois Urbana-Champaign
Urbana, Illinois, USA
sean@isper.net

Abstract

Scientific literature review often requires comparing many papers that appear to agree, disagree, or provide mixed evidence about the same research topic. This process is especially difficult when claims are distributed across abstracts, use different experimental settings, or report different metrics. This project introduces *Contradiction Finder for Research Reading*, a Streamlit-based web application that helps users discover possible contradictions or tensions across research paper abstracts. Given a user-entered topic, the system retrieves papers from Semantic Scholar, extracts and normalizes scientific claims, constructs semantically related cross-paper claim pairs, scores possible contradictions using a local Natural Language Inference (NLI) model, and generates evidence-grounded conflict cards using a Groq-hosted large language model. The tool is designed as a transparent reading assistant rather than an automatic fact checker. The system also saves intermediate JSONL artifacts and includes evaluation scripts for SciFact benchmark testing and manual card labeling. On a SciFact validation sample of 200 examples, the local NLI scorer achieved 42.5% overall accuracy and 0.57 F1 on the contradiction class, showing that the system is useful for candidate discovery but still requires cautious human interpretation.

Keywords

literature review, contradiction detection, scientific claims, information retrieval, natural language inference, Streamlit, SciFact

1 Introduction

Literature review is a central task for students, researchers, and professionals who need to understand the state of evidence in a research area. However, literature review is increasingly challenging because the number of available papers continues to grow, and because papers often make claims that appear to conflict with each other. A researcher may find one paper reporting that a method reduces hallucination, another paper warning that the same method is vulnerable under adversarial conditions, and a third paper reporting a very different improvement percentage. Determining whether these claims are truly contradictory requires manually comparing datasets, methods, assumptions, evaluation metrics, and experimental conditions.

This project addresses that problem by building a web-based literature review assistant called *Contradiction Finder for Research Reading*. The system is designed to help users discover, inspect, and understand possible conflicts across a set of scientific papers. The intended users are students conducting class projects, researchers entering a new field, and practitioners who need to quickly understand where papers agree, disagree, or provide mixed evidence.

The project does not attempt to automatically determine which paper is correct. Instead, it treats contradiction detection as a reading-support task. The system surfaces possible tensions, connects them to the original evidence snippets, and gives the user enough context to decide whether the disagreement is meaningful. This design choice is important because scientific contradictions are rarely simple sentence-level negations. Many apparent contradictions are better understood as differences in dataset, metric, model, population, task, or condition.

The main contributions of this project are:

(1) A complete end-to-end Streamlit application for contradiction-aware literature review over paper abstracts.
(2) A transparent pipeline that saves every intermediate artifact: retrieved papers, sentences, claims, candidate pairs, NLI scores, conflict cards, and run statistics.
(3) A modular implementation using Semantic Scholar retrieval, sentence preprocessing, claim extraction, embedding-based pairing, local NLI scoring, Groq-based explanation generation, and downloadable outputs.
(4) An evaluation layer with SciFact benchmark testing and manual-labeling utilities for estimating generated conflict cards quality.

2 Motivation and Intended Users

The motivation behind this project is that literature review can be time-consuming and cognitively demanding. Traditional search tools return lists of papers, but they rarely organize papers by claims, evidence, or disagreement. Users still need to read abstracts one by one, copy relevant claims into notes, compare results, and decide whether apparent disagreements are real contradictions or just differences in experimental setup.

The intended users are:

- **Students** performing literature reviews for course projects.
- **Researchers** entering a new area and trying to understand the structure of evidence.
- **Professionals and practitioners** who need to quickly inspect mixed findings before making technical decisions.

The significance of the tool is that it transforms a flat list of retrieved papers into a structured view of claims and possible conflicts. A contradiction-aware assistant can reduce manual cross-checking time, encourage more critical reading, and help users avoid overconfident conclusions based on a narrow subset of evidence.

3 System Overview

Contradiction Finder follows a multi-stage pipeline. The user enters a topic query and configures run settings in the Streamlit sidebar. The system then retrieves papers from Semantic Scholar, extracts

and normalizes claims, pairs semantically related claims across papers, scores the pairs with a local NLI model, and displays high-scoring possible contradictions as conflict cards.

The pipeline can be summarized as:

```
User topic query
  -> Semantic Scholar paper retrieval
  -> Abstract cleaning and sentence splitting
  -> Claim candidate extraction
  -> Groq validation, normalization, and claim recovery
  -> SentenceTransformer claim embeddings
  -> Cross-paper candidate pair construction
  -> Local NLI contradiction scoring
  -> Groq conflict explanation and reason categorization
  -> Streamlit tables, conflict cards, and exports
```

The application uses abstracts only. It does not download or parse full PDFs, which keeps the MVP tractable, reproducible, and appropriate for a course-scale development project.

3.1 Major Functions

The major functions of the system are:

- **Paper retrieval:** Search Semantic Scholar by topic and retrieve metadata and abstracts.
- **Caching:** Save Semantic Scholar responses locally by query hash to support reproducibility and faster reruns.
- **Sentence splitting:** Convert abstracts into traceable sentence-level records.
- **Claim extraction:** Identify claim-like sentences using transparent heuristics and Groq-based validation/recovery.
- **Claim pairing:** Use sentence embeddings and cosine similarity to identify related claims across different papers.
- **Contradiction scoring:** Use a local Hugging Face NLI model to score entailment, contradiction, and neutral relationships.
- **Conflict explanation:** Generate a short title, reason category, and cautious interpretation for each conflict card.
- **Interactive UI:** Provide metrics, tables, conflict cards, paper links, and downloads through Streamlit.
- **Evaluation:** Run SciFact benchmark evaluation and export manual-labeling CSV files.

4 Technical Architecture

The implementation is modular. The Streamlit app is intentionally thin and calls a pipeline controller rather than containing all business logic. The source code is organized into separate modules:

```
app.py
scripts/
  evaluate_scifact.py
  export_manual_labels.py
  evaluate_manual_labels.py
src/
  config.py
  schemas.py
  retrieval/semantic_scholar.py
  preprocessing/sentence_splitter.py
  claims/
  embeddings/
  clustering/
  nli/
  cards/
  evaluation/
  pipeline/run_pipeline.py
```

The most important architectural principle is traceability. Each final card can be traced backward from card to NLI score, candidate pair, normalized claim, original abstract sentence, and source paper. This is critical because the system is intended to support human reading rather than replace human judgment.

4.1 Data Models

The project uses Pydantic schemas for all intermediate artifacts. The key models are:

- **Paper:** metadata and abstract returned by Semantic Scholar.
- **PaperSentence:** cleaned abstract sentence with paper ID and sentence index.
- **Claim:** normalized scientific claim with original sentence, claim type, extraction method, and confidence.
- **ClaimPair:** cross-paper pair of semantically related claims.
- **ContradictionScore:** entailment, contradiction, neutral scores, and predicted NLI label.
- **ConflictCard:** card with both claims, evidence snippets, paper metadata, score, reason category, and explanation.

All intermediate outputs are serializable to JSONL, which makes the pipeline easier to debug and evaluate.

4.2 External Models and APIs

The project uses several external components:

- **Semantic Scholar Academic Graph API** for paper metadata and abstracts [1].
- **Groq API** for structured JSON claim validation, abstract-level claim recovery, and conflict-card explanations [2].
- **SentenceTransformer model:** `sentence-transformers/all-MiniLM-L6-v2` for claim embeddings [3].
- **Hugging Face NLI model:** `cross-encoder/nli-deberta-v3-small` for local contradiction scoring [4].

Semantic Scholar and Groq require API access. The app can run without a Semantic Scholar key using unauthenticated access. If the Groq key is missing, the app falls back to heuristic-only claims and generic fallback explanations.

5 Key Algorithms

5.1 Claim Extraction

The claim extraction process combines transparent heuristics with optional LLM validation. The heuristic extractor first selects sentences that contain claim-like patterns such as “we find”, “results show”, “improves”, “reduces”, “outperforms”, “fails to”, or “leads to”. It also removes obvious paper-structure sentences such as “In this paper, we propose...” unless they state an effect or result.

The selected candidate sentences are then passed to Groq for structured JSON validation and normalization. Groq determines whether each sentence expresses a concrete scientific claim and returns a concise standalone version of the claim. When enabled, the system also asks Groq to recover up to three key claims from each abstract, which improves recall when heuristic patterns miss important findings.

Pseudo code:

```
function extract_claims(papers, sentences, use_groq,
      recover_missing):
```

```
    candidates = heuristic_claim_candidates(sentences)
    claims = []

    if use_groq is false or Groq client is unavailable:
        return heuristic_only_claims(candidates)

    for sentence in candidates:
        paper = lookup_paper(sentence.paper_id)
        claim = groq_validate_and_normalize(sentence, paper)
        if claim is not null:
            claims.append(claim)

    if recover_missing:
        for paper in papers:
            recovered = groq_recover_claims_from_abstract(paper,
      max_claims=3)
            claims.extend(recovered)

    return deduplicate_claims(claims)
```

5.2 Candidate Pair Construction

After claims are extracted, the system embeds each normalized claim with a SentenceTransformer model. Since comparing every possible claim pair can be expensive and noisy, the system keeps only cross-paper pairs whose cosine similarity is above a user-selected threshold. It also limits the number of pairs per claim to avoid all-pairs explosion.

Pseudo code:

```
function build_candidate_pairs(claims, embeddings, threshold,
     max_pairs_per_claim):
    similarities = embeddings dot embeddings.T
    pairs = []
    seen = empty set

    for each claim i:
        ranked_neighbors = sort_by_similarity_descending(
      similarities[i])
        kept = 0

        for each candidate j in ranked_neighbors:
            if i == j:
                continue
            if claims[i].paper_id == claims[j].paper_id:
                continue
            if similarity(i, j) < threshold:
                break

            pair_key = sorted(claim_i_id, claim_j_id)
            if pair_key in seen:
                continue

            add pair to pairs
            add pair_key to seen
            kept += 1

            if kept >= max_pairs_per_claim:
                break

    return pairs sorted by similarity
```

5.3 Contradiction Scoring

Each candidate pair is scored using the local NLI model. The scorer runs the model in both directions: claim A as premise and claim B as hypothesis, then claim B as premise and claim A as hypothesis. This is useful because NLI models can be direction-sensitive. The system combines the two directions by taking the maximum score for each label.

Pseudo code:

```
function score_pair(pair):
    forward = NLI(pair.claim_a, pair.claim_b)
    reverse = NLI(pair.claim_b, pair.claim_a)

    combined.entailment = max(forward.entailment, reverse.
      entailment)
    combined.contradiction = max(forward.contradiction, reverse.
      contradiction)
    combined.neutral = max(forward.neutral, reverse.neutral)

    predicted_label = argmax(combined)
    return ContradictionScore(pair_id, combined scores,
      predicted_label)
```

The conflict-card generator keeps pairs whose predicted label is contradiction and whose contradiction score is above the selected threshold.

5.4 Conflict Explanation

For each high-confidence possible contradiction, Groq generates: (1) a short topic title, (2) a reason category, such as `different_dataset`, `different_metric`, `different_method`, `different_condition`, or `unclear_from_abstract`, and (3) a cautious one- or two-sentence interpretation.

The prompt instructs the model to use only the provided claims and evidence snippets, avoid inventing missing details, and avoid saying that one paper is correct and the other is wrong.

6 User Interface and Usage

The user interface is implemented in Streamlit [6]. The sidebar contains all run settings:

- Research topic input.
- Paper limit slider from 10 to 50.
- Similarity threshold slider from 0.30 to 0.80.
- Contradiction threshold slider from 0.50 to 0.90.
- Maximum number of cards slider from 5 to 30.
- Use cached retrieval checkbox.
- Use Groq checkbox.
- Recover missing claims checkbox.
- Run pipeline button.

After the pipeline completes, the app displays summary metrics and a segmented view selector.

6.1 Conflict Cards View

The conflict cards view is the main output of the system. Each card includes a "Possible contradiction" badge, a topic title, NLI contradiction score, reason category, side-by-side claims, original evidence sentences, source paper titles and years, paper links, and a cautious interpretation.

For example, in the demo query `retrieval augmented generation hallucination reduction`, the system produced a card titled *RAG System Vulnerability Disagreement*. One claim stated that injecting poisoned text into the knowledge database can compromise RAG systems, while the other stated that combining RAG with zero-shot prompting creates trustworthy summarization tools. The generated reason category was `different_condition`, indicating that the apparent disagreement may depend on whether the system is being evaluated under normal or adversarial conditions.

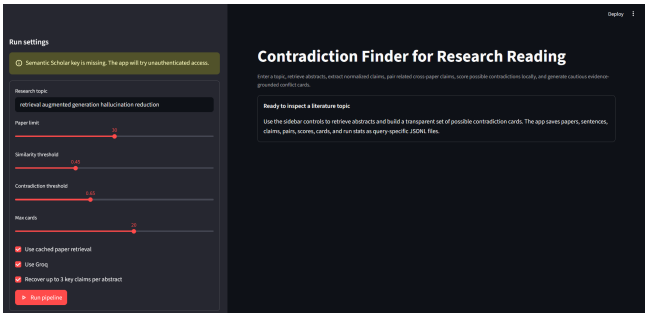

Figure 1: Main Streamlit interface with run settings and summary metrics.

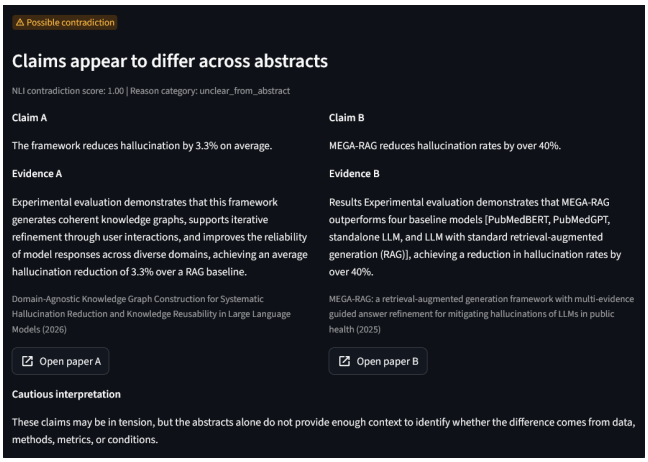

Figure 2: Example conflict card showing two claims, evidence snippets, reason category, and cautious interpretation.

6.2 Retrieved Papers View

The retrieved papers view shows paper title, year, venue, authors, citation count, and URL. A toggle allows the user to inspect abstracts directly. This view is useful for verifying whether the retrieval step returned relevant papers.

6.3 Extracted Claims View

The extracted claims view shows each normalized claim, source paper, claim type, confidence score, extraction method, and original sentence. This view helps users inspect whether the claim extraction step is reasonable and grounded in the abstract.

6.4 Candidate Pairs View

The candidate pairs view shows claim A, claim B, semantic similarity, NLI contradiction score, NLI label, source papers, and original evidence. This view is useful for debugging or threshold tuning because it shows pairs before they become final conflict cards.

Table 1: Pipeline counts from the RAG hallucination demo run.

Stage	Count
Retrieved papers	29
Abstract sentences	225
Extracted claims	120
Candidate pairs	373
Scored pairs	373
Possible contradictions	33
Final conflict cards	20

6.5 Exports View

The exports view lets users download all saved artifacts. It also provides a manual-labeling CSV export for evaluating generated conflict cards. The downloadable files include `papers.jsonl`, `sentences.jsonl`, `claims.jsonl`, `pairs.jsonl`, `scores.jsonl`, `cards.jsonl`, `stats.json`, and the manual-labeling CSV.

7 Demo Run

The main demo query used for this project was:

```
retrieval augmented generation hallucination reduction
```

Using the default settings, one run produced the pipeline counts shown in Table 1.

The demo output illustrates the kind of scientific tensions the system is designed to surface. Some cards compare different hallucination reduction percentages, such as 3.3%, 40%, 54.1%, or 71%. Others compare positive claims about RAG improving factuality with negative claims about RAG vulnerability to poisoning attacks. These examples are useful, but they also show why the system uses cautious reason categories: many apparent contradictions are actually differences in dataset, method, baseline, metric, or condition.

8 Evaluation

The evaluation layer has two parts: automatic benchmark evaluation using SciFact and manual review of generated conflict cards.

8.1 SciFact Evaluation

SciFact is a scientific fact verification dataset containing claims, evidence abstracts, labels, and rationale sentence indices [5]. Although SciFact is not exactly the same task as open-ended contradiction discovery, it provides a controlled way to test the local NLI scorer on scientific text.

The evaluation script converts SciFact examples into claim-evidence pairs and runs the same local NLI model used by the app. It then compares predicted labels against gold labels. The labels used in evaluation are contradiction, entailment, and neutral. The script also computes grounding metrics to verify that evidence text is correctly connected to the intended document and sentence indices.

On a validation sample of 200 examples, the local NLI scorer produced the results in Table 2.

The confusion matrix was as shown in Table 3.

These results show that contradiction detection is stronger than entailment detection in this setup. The contradiction class achieved

Table 2: SciFact validation evaluation results on 200 examples.

Metric	Value
Evaluated pairs	200
Overall accuracy	0.425
Macro F1	0.404
Weighted F1	0.348
Contradiction precision	0.641
Contradiction recall	0.521
Contradiction F1	0.575
Rationale grounding accuracy	1.000

Table 3: SciFact confusion matrix. Rows are gold labels and columns are predicted labels.

Gold / Predicted	Contradiction	Entailment	Neutral
Contradiction	25	0	23
Entailment	9	7	78
Neutral	5	0	53

Table 4: Threshold sweep for contradiction prediction.

Threshold	Precision	Recall	F1	Pred. Contr.
0.50	0.526	0.625	0.571	57
0.65	0.549	0.583	0.566	51
0.80	0.574	0.563	0.568	47
0.90	0.634	0.542	0.584	41

0.57 F1, while entailment recall was low. This is acceptable for the project goal because the app is primarily designed to surface candidate contradictions, but it also confirms that the system should not be treated as an automatic truth evaluator.

8.2 Threshold Sweep

The SciFact evaluation also tested contradiction thresholds from 0.50 to 0.90. The best contradiction F1 in the sample occurred at threshold 0.90, as shown in Table 4.

This suggests that the default threshold of 0.65 is useful for exploratory discovery, while a higher threshold such as 0.85 or 0.90 may be better when users want fewer false positives.

8.3 Manual Evaluation Workflow

The system also includes a manual evaluation workflow. The app can export generated conflict cards to a CSV file with columns for the card contents, manual label, and reviewer notes. Reviewers can label each card as `true_contradiction`, `partial_or_contextual_contradiction`, `not_contradiction`, or `unclear`.

The manual metrics script then computes strict precision, broad precision, precision excluding unclear cases, and unclear rate. For the RAG hallucination demo run, the manual CSV contained 20 conflict cards. At the time of writing, those rows had not yet been

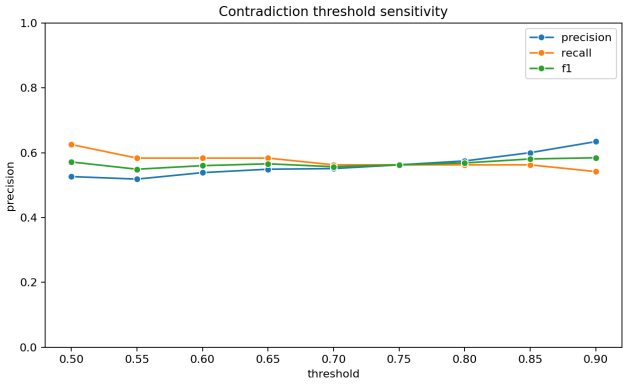

Figure 3: Threshold sweep plot from `data/evaluation/scifact/validation/threshold_sweep.png`.

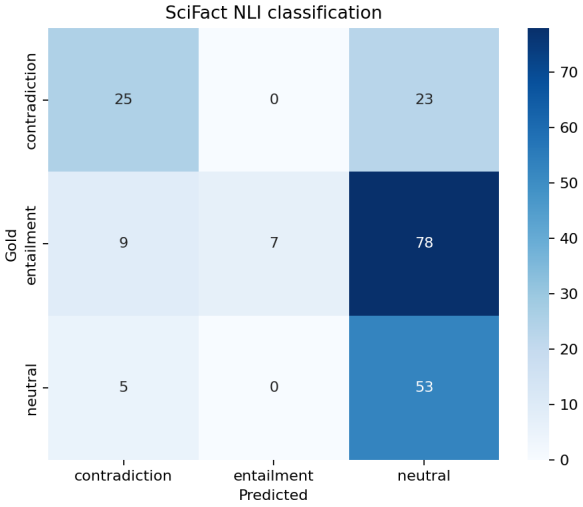

Figure 4: Confusion matrix plot from `data/evaluation/scifact/validation/confusion_matrix.png`.

manually labeled, so the manual evaluation layer is implemented as a review workflow rather than final precision results. This workflow is still important because many scientific disagreements require human interpretation beyond sentence-level NLI scoring.

9 Discussion

The project demonstrates that a modular pipeline can turn a literature topic into a structured set of possible scientific disagreements. The system works best as an exploratory reading assistant. It is useful for identifying where to look more closely, especially when different papers report different improvement percentages or evaluate a method under different assumptions.

At the same time, the results show the difficulty of scientific contradiction detection. NLI models are sensitive to surface-level polarity. For example, a claim that a method improves hallucination rates may be flagged as contradictory to a claim that the method is

vulnerable to poisoning, even though both claims can be true under different conditions. This is why the system includes reason categories such as `different_condition`, `different_method`, and `different_metric`. These categories help users interpret apparent contradictions more carefully.

10 Limitations

The main limitations are:

(1) **Abstract-only evidence.** The system does not download or parse full papers, so important details from methods, experiments, or results sections may be missing.
(2) **NLI false positives.** The local NLI model can over-detect contradictions when two claims use opposite-sounding language but refer to different conditions.
(3) **LLM explanation uncertainty.** Groq-generated explanations are grounded in provided claims and evidence snippets, but they may still infer reason categories imperfectly.
(4) **API and model dependencies.** Semantic Scholar retrieval, Groq explanations, and Hugging Face model downloads require network access.
(5) **Limited benchmark match.** SciFact is useful for evaluation, but not same as open-ended literature contradiction discovery.
(6) **No full user study.** The project includes benchmark and manual-evaluation, but it doesn't include a formal user study.

11 Future Work

Future work could improve the project in several directions:

- Add full-text PDF parsing to access methods, experiments, and results sections.
- Use a stronger scientific NLI or claim verification model.
- Add a second-stage adjudicator that distinguishes direct contradictions from contextual differences.
- Cluster cards by topic so users can inspect groups of related disagreements.
- Add citation graph context, such as whether later papers cite or correct earlier findings.
- Complete manual labeling for several topics and report human acceptance rates.
- Add user-study evaluation to measure whether the tool reduces literature review time.

12 How to Use the Software

To use the software, clone the GitHub repository:

```
git clone https://github.com/sliao082/Contradiction-Finder.git
cd Contradiction-Finder
```

Create and activate a Python environment:

```
python -m venv .venv
.\.venv\Scripts\Activate.ps1
pip install -r requirements.txt
```

Create a `.env` file:

```
Copy-Item .env.example .env
```

Then add:

```
GROQ_API_KEY=your_groq_api_key_here
SEMANTIC_SCHOLAR_API_KEY=your_semantic_scholar_api_key_here
GROQ_MODEL=llama-3.3-70b-versatile
GROQ_API_URL=https://api.groq.com/openai/v1/chat/completions
EMBEDDING_MODEL=sentence-transformers/all-MiniLM-L6-v2
NLI_MODEL=cross-encoder/nli-deberta-v3-small
```

Run the Streamlit app:

```
streamlit run app.py
```

Open the local URL printed by Streamlit, usually:

```
http://localhost:8501
```

Recommended first query:

```
retrieval augmented generation hallucination reduction
```

Recommended default settings:

```
Paper limit: 30
Similarity threshold: 0.45
Contradiction threshold: 0.65
Max cards: 20
Use cached paper retrieval: enabled
Use Groq: enabled
Recover up to 3 key claims per abstract: enabled
```

To run the SciFact evaluation:

```
python scripts/evaluate_scifact.py --split validation --max-
    examples 200
```

To export manual labels from a saved run:

```
python scripts/export_manual_labels.py data/processed/runs/{
    query_hash}
```

To evaluate completed manual labels:

```
python scripts/evaluate_manual_labels.py data/evaluation/manual/{
    query_hash}_manual_labels.csv
```

13 Conclusion

Contradiction Finder for Research Reading provides a practical web-based assistant for inspecting possible disagreements across scientific paper abstracts. The system retrieves papers, extracts claims, pairs related claims, scores possible contradictions, generates cautious explanations, and presents results in a transparent interface. The evaluation results show moderate contradiction-detection performance and motivate careful human review, which matches the tool's intended role as a reading assistant rather than a fully automatic fact checker. Overall, the project demonstrates a substantial end-to-end implementation for transforming a literature topic into a structured view of claims, tensions, and supporting evidence.

References

[1] Semantic Scholar. 2026. Semantic Scholar Academic Graph API. Retrieved May 12, 2026 from https://api.semanticscholar.org/api-docs/
[2] Groq. 2026. Groq API Documentation. Retrieved May 12, 2026 from https://console.groq.com/docs/
[3] Sentence Transformers. 2026. sentence-transformers/all-MiniLM-L6-v2. Retrieved May 12, 2026 from https://huggingface.co/sentence-transformers/all-MiniLM-L6-v2
[4] Hugging Face. 2026. cross-encoder/nli-deberta-v3-small. Retrieved May 12, 2026 from https://huggingface.co/cross-encoder/nli-deberta-v3-small
[5] Allen Institute for AI. 2026. SciFact Dataset. Retrieved May 12, 2026 from https://huggingface.co/datasets/allenai/scifact
[6] Streamlit. 2026. Streamlit Documentation. Retrieved May 12, 2026 from https://docs.streamlit.io/